ENCM 511: Embedded System Interfacing
Assignment 3

Group 6
Chloe Fulbrook - 30210975
Ethan Lee - 30213225
Devon Calvin - 30204532

Submitted on October 24th, 2025

General Code Explanation:

For Lab 3, our code is generally organized into a main file and three headers/source files. This helps us create our functions separately from main and assists with readability.

init.c
This file is essentially identical to the file submitted in the last lab. The only changes are essentially modifying the IOC interrupts for each button to interrupt on both positive and negative edges. This file simply initializes the IOs, so the buttons and LEDs, and configures the IOC interrupts. It also sets up the timers, and creates a delay_ms function, although this function is not used in our final implementation.

init.h
This is again essentially the same as in the last lab. It creates macros for the buttons and leds, and holds the function prototypes defined in init.c.

uart.c
This is essentially the same as what was given to us as part of the lab package. We added a null terminator for the string in RecvUart() to assist with the assignment part, added an escape via the PB2 flag in RecvUartChar() to help with the PB2 in prog mode, and also created a function to clear the terminal using escape codes.

uart.h
This is also basically unchanged from what was given in the lab package. We added the function prototype for the clear_terminal() function that we created, and pulled the PB2 flag for use as well.

transitions.c
This holds our defined functions controlling our state transitions for the finite state machine. We start by including the necessary header files for use, and then we initialize the externally defined flags to zero. The first function, to_idle, takes parameters of pointer-to-flag and prog. The flag that is triggering the transition to idle is passed as a pointer in order to help with the clearing and streamlining of code. Prog is passed in order to keep track of whether we are in fast mode or prog mode. We first clear the flag by setting the pointer to 0, and we turn off the LED as the idle states should have all LEDs off. We also turn off the timer to stop any interrupts and potential LED blinking. Logic then checks if the machine is in prog mode or fast mode, clears the terminal, and then appropriately prints a message depending on which mode it is in. Finally, the function returns STATE_idle in the state_t type to cause transition in the main() FSM.

The next function is the single_press_transition function, which takes parameters of pointer-to-flag and prog, same as the to_idle function. Starts by defining a temporary variable

temp_state to keep track of the necessary state transition in the function overall. It then checks if the mode is not prog mode, and proceeds to handle logic for state transitions. All of the single-button states in fast mode have an LED blinking, so we begin by setting the LED on to allow for a cleaner look. We also set a variable to be used to help with the UART printing of the clicked button. Next, logic checks for which button has been clicked by seeing if the pointer-to-flag passed as a parameter points to the same place as the address of each button's flag itself. Each button works basically the same, where the button number is saved as a char in the btn variable for use in the UART display, sets the appropriate blink rate as defined by the lab requirements, and stores the state of transition in the temp_state variable to cause FSM transition upon returning to main(). After this logic, the terminal is cleared, and then the Fast mode message is printed with the appropriate button. The start count for the timer is then set, timer is turned on to start the blinking, the flag is cleared via the pointer, and the stored state for transition is returned to main().

Still in single_press_transition, we then look at if the machine is in prog mode. The button causing the transition is again checked using the flag pointer and associated addresses. For PB0, we clear the terminal and display the appropriate prog mode message, set the timer high count to a 3 second blink rate, and store the STATE_PB0 for transition in the temp_state variable. For PB1, we clear the terminal and display the appropriate message, also displaying the user-entered mode from the PB2 version of this. The timer 3 high count is set to that variable, user-entered rate as well (0.25s, 0.5s, 1s), and the STATE_PB1 is stored in temp_state for transition in main(). The PB2 is slightly more complicated. We again start by clearing the terminal and displaying the PB2 prog mode message, setting the timer high count to 0.125s and turning on timer 3 to blink the LED while waiting for user input. We also must clear the pointer-to-flag for this one, as we need to wait for PB2 to be clicked again to save the entered mode. We set a char rec to hold the user mode, and call RecvUartChar() to get input over UART. Then, if PB2 is pressed, we use the variable rec to determine what the blink rate should be when put eventually into the prog mode PB1. We also store the char rec in a variable that is later used to do UART output. We then clear the terminal, display the prog mode idle, and set the temp_state to STATE_IDLE for main() transition. The timer is then turned on, the pointer-to-flag is cleared, and we return to main with the stored temp_state.

The final function in transitions.c is the dual_press_transition() function. This handles any transitions that rely on two buttons having been pressed. We start by turning the LED on, as all of these states should have the LED constantly on, and we turn off the timer so that we do not see toggling of the LED or interrupting. We create a small char array to help with the UART display, and then check which combination of buttons was pressed to cause the transition via the pre-set flags. Each condition in the if/else logic works the same, with the appropriate flag being cleared, and the pb_nums array being set to the correct values. Then we clear the terminal and display the appropriate message depending on which buttons were pressed, and we return to main() with STATE_dual to cause transition.

transitions.h
This is the header file accompanying the transitions.c file. It holds all of our extern volatile flags and variables, as they are accessed in multiple files by multiple functions. Note that they are only declared, but not actually initialized here. We also define our blink rate macros for all of the possible necessary blink rates in the project. The states for the state machine are created, and the state_t type is also defined. Finally, it houses the function prototypes for the functions created in transitions.c.

lab3_main.c
The main file holds the bulk of our code logic. It begins with the #pragmas for setup and configuration, then includes the necessary header files. We then create a number of variables that are used to help with our button logic in the timer2 ISR. These are the last valid (_LV), before debounce (_BD), and after debounce (_AD) statuses of the buttons. Upon entering the primary main(), we configure the pins as digital, call our initializer functions from init.c, and default the state machine to STATE_IDLE.

We then enter the superloop while(1), where the switch statement for the finite state machine is held. When entering any case, we force the CPU into Idle() for power-saving purposes. State transitions are handled by the flags set by the IOC and TMR2 ISRs. The idle state is the only one where the three-press can cause a transition, so this is handled explicitly in this case. If the trip_flag is set, we clear it and toggle the prog state, then display the appropriate mode's idle message. Otherwise, in the idle state along with the single-button states, we check the button flags and call the single_press_transition function with the associated flag. For the dual presses, we check if any of the dual flags are set and call dual_press_transition if we are in fast mode. If in prog mode, we ignore the dual flags and clear them. For the single-button states, we also check if the button corresponding to the state that we are in was pressed, where we then return to idle via the to_idle function. The STATE_dual is similar, except we cannot transition to a single-button state from this one. Any single press calls to_idle with the appropriate flag, and clears all of the dual flags. If a different set of two flags is pressed, the dual_press_transition is called, and the UART display is updated accordingly.

The main file then progresses to its ISRs. As discussed in the last lab, there is only one IOC ISR on our microcontroller, so any button event is handled by one ISR. The IOCs on the button pins are configured to interrupt on both positive and negative edges in this lab. Upon entering the ISR, we check which pin caused the interrupt and set the _BD flag for that button. This tracks the state of the button immediately as the event occurred, and is useful for the debouncing of the buttons. We also set up timer 2 to be used as a debounce counter, setting its start count to 0, the high count to ~50ms enabling its interrupts, turning it on, and then clearing the button's IOC flag. This leads us to the timer 2 ISR.

The timer 2 ISR is more complicated with the button logic that we are required to detect in this lab. Upon entering the ISR, we clear the timer interrupt flag, disable the interrupt, and turn off

the timer. We then check if each button's current state is the same as the state that was saved immediately upon entering the IOC ISR. If yes, we set the _AD (after debounce) value to the current state of the button, and if no, we default the _AD to 1 because of our button configuration. After this, we proceed to check the button combinations in order to set the appropriate flags for use in the state transitions in main. If all of the buttons have a current _AD value that is a logic low and the last valid _LV a logic high, all of the buttons are pressed and we set the trip_flag. Else, if the same logic has occurred but only on two of the buttons, we set the appropriate dual_flag_xx flags. Finally, if a button has a _AD of logic 1 and a _LV of logic 0, we have seen a low to high transition, saying that the button has been released. But, this is not enough to stabilize the logic, as this causes unnecessary transitions when releasing the buttons during the trip or dual button presses. So, we check to see if the other buttons have been stable while the button being examined has made its transition. We are looking to see that they have a current _AD of 1 and _LV of 1 as well, saying that they have not just transitioned. This is what allows us to differentiate the click from the press for the buttons, where the click must be defined after the button has both been pressed and released. After all of the combination conditions have been checked, we set the _LV of each button to its current _AD, for later comparison.

Finally, we have the timer 3 ISR, which is used to control the blinking of the LED. This is a simple ISR, where all we do is clear the interrupt flag and then toggle the LED on each interrupt to create the blinking.

Special Function(s):

Clear_terminal

```
void clear_terminal (void) {
   Disp2String("\033[2J\033[H");
}
```

This contains the ANSI escape codes to clear the terminal screen and reset the cursor to the top left. The purpose of this function is to clear the terminal when we want to display different text. Putting this before any call of Disp2String ensures that the terminal always only displays one line.

To_idle

```
state_t to_idle(volatile uint16_t *flag, volatile uint16_t prog){
// this function is used to transition the state machine to idle from
whichever state it is in
```

```
    *flag = 0;  // clear the flag for the button being used to transition
to idle
    LED0 = 0;  // turn the LED off
    T3CONbits.TON = 0;      // turn off timer
    if(!prog){
// if not in prog mode, clear terminal and display the idle fast mode
        clear_terminal();
        Disp2String("\rFast Mode: Idle");
    }
    else if(prog){
// if in prog mode, clear terminal and display the idle prog mode
        clear_terminal();
        Disp2String("\rProg Mode: Idle");
    }
    return STATE_IDLE; // put the state machine into the idle state upon
return
}
```

The purpose of this function is to return the FSM back to the idle state. It resets the IOC flag that causes the transitions and will display idle for the mode it is in. This was created to simplify the look of the code as many different combinations of button presses return the FSM to idle.

single_press_transition

```
state_t single_press_transition(volatile uint16_t *flag, volatile uint16_t
prog){
state_t temp_state;
if(!prog){
    LED0 = 1;
    char btn;
    if(flag == &PB0_flag){
        btn = '0';
        // initialize timer
        PR3 = blink_025;
        temp_state = STATE_PB0;
    }
    else if(flag == &PB1_flag){
```

```
        btn = '1';
        // initialize timer
        PR3 = blink_05;
        temp_state = STATE_PB1;
    }
    else if(flag == &PB2_flag){
        btn = '2';
        // initialize timer
        PR3 = blink_1;
        temp_state = STATE_PB2;
   }
    clear_terminal();
    Disp2String("\rFast Mode: PB");
    XmitUART2(btn, 1);
    Disp2String(" was pressed");
    TMR3 = 0;
    T3CONbits.TON = 1;      // turn on timer
    *flag = 0;
    return temp_state;
}
else if(prog){
    if(flag == &PB0_flag){
        clear_terminal();
        Disp2String("\rProg Mode: PB0 was pressed");
        // initialize timer
        PR3 = blink_3;
        temp_state = STATE_PB0;
    }
    else if(flag == &PB1_flag){
        clear_terminal();
        Disp2String("\rProg Mode: PB1 was pressed, Setting = ");
        XmitUART2(prog_mode_X, 1);
        PR3 = prog_PB1_blink;
        temp_state = STATE_PB1;
    }
    else if(flag == &PB2_flag){
        clear_terminal();
        Disp2String("\rProg Mode: Blink Setting = ");
        PR3 = blink_0125;
        TMR3 = 0;
```

```
        T3CONbits.TON = 1;
        *flag = 0;          // check this
        char rec = RecvUartChar();
        if(PB2_flag){
            if(rec == '0'){
                prog_PB1_blink = blink_025;
            }
            else if(rec == '1'){
                prog_PB1_blink = blink_05;
            }
            else if(rec == '2'){
                prog_PB1_blink = blink_1;
            }
            prog_mode_X = rec;
            clear_terminal();
            Disp2String("\rProg Mode: Idle");
            temp_state = STATE_IDLE;
            PB0_flag = 0;
            PB1_flag = 0;
        }
    }
    TMR3 = 0
    T3CONbits.TON = 1;
    *flag = 0;
    return temp_state;
}
}
```

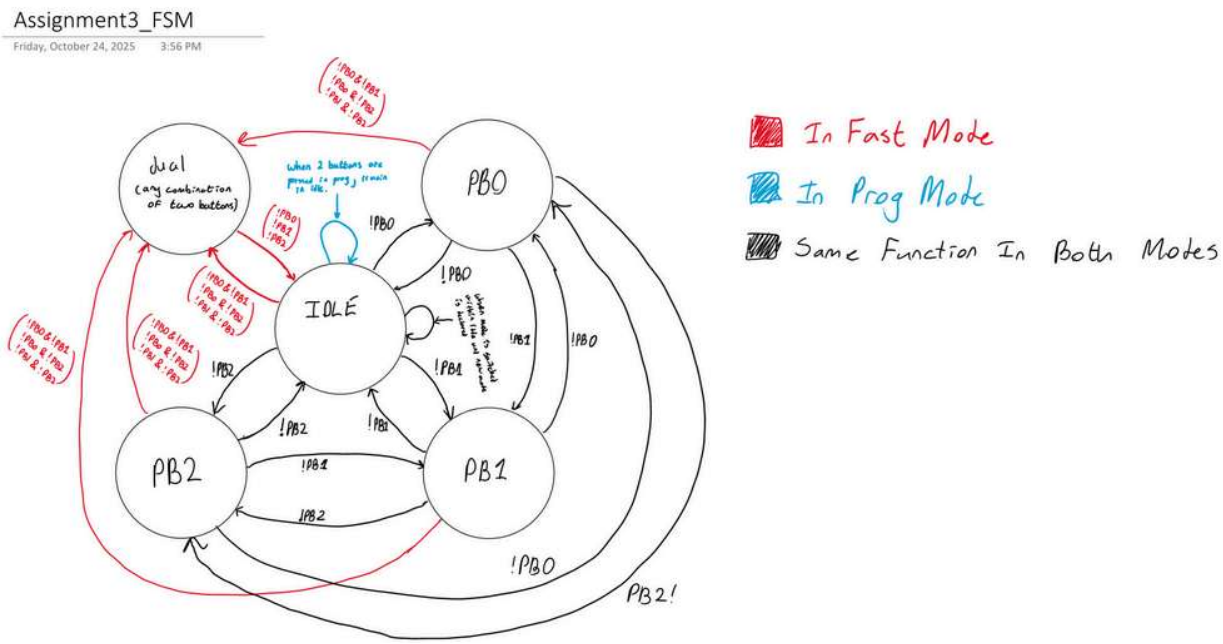
The function is for many of the other state transitions that only use a single button press. The other transitions were made into a function to make the main code look cleaner A general explanation is that depending on if the flag for prog mode is set off, it will execute the single button press code for prog or fast mode. A description of how the code within the function executes was described in the general code explanation.

Dual_press_transition

```
state_t dual_press_transition(){
    LED0 = 1;
    T3CONbits.TON = 0;
    char pb_nums[2];
    if(dual_flag_01){
        dual_flag_01 = 0;
        pb_nums[0] = '0';
        pb_nums[1] = '1';
    }
    else if(dual_flag_02){
        dual_flag_02 = 0;
        pb_nums[0] = '0';
        pb_nums[1] = '2';
    }
    else if(dual_flag_12){
        dual_flag_12 = 0;
        pb_nums[0] = '1';
        pb_nums[1] = '2';
    }
    clear_terminal();
    Disp2String("\rFast Mode: PB");
    XmitUART2(pb_nums[0], 1);
    Disp2String(" and PB");
    XmitUART2(pb_nums[1], 1);
    Disp2String(" are pressed");
    return STATE_dual;
}
```

The function is for the dual button presses. Instead of writing the code in each state, it was turned into a function to make the main code look cleaner. It displays different text on the terminal depending on which combination of two buttons were pressed. Button combinations will set pb_nums[0], pb_nums[1] to the buttons that were pressed. This is then used later to display what combination of buttons were pressed on the terminal.

Our Moore-type FSM:



Questions:

(i) What was the target baud rate? What is the % error of the actual baud?

High Baud Rate is enabled and so the actual baud rate is calculated to be 9615.384615 bits per second. The target baud rate for our terminals was 9600 bps. There exists a 0.156% error for that value.

(ii) What are your comments/explanations for XmitUART2?

For line 60, while(U2STAbits.UTXBF==1), the purpose of this loop is to wait until the transfer buffer is not full. This is important to make sure data that is sent to the transmit buffer register does not overwrite data that is waiting to be sent.

For line 63 and 64, U2TXREG=CharNum; and repeatNo--;. The U2TXREG is a register that we write the data we want to be transmitted. It exists after line 60 so that the data we want to send does not overwrite any values waiting to be sent in the buffer. The repeatNo-- exists so that the same data can be sent multiple times. It's important to note that the repeatNo while loop always executes while(U2STAbits.UTXBF==1) for the reasons above.

Line 66, while(U2STAbits.TRMT==0){}, ensures that all the data has been sent before turning off the transmitter. This makes sense as TMRT equals 1 when the transmit buffer is empty and 0 when it is not. If it's not empty then a transmission is in progress or queued.

(iii) In your assignment part, how do you get the LED to blink while waiting for a character to be input? Is the character receive function a “busy wait,” i.e., does it stop/block the CPU from doing something else?

The blink rate exists within our Timer3 interrupt. The PR3 value and other timer 3 configurations are set upon entering the PB2 state in prog mode. The character receive function is a busy wait function as it will keep accepting characters due to the while loop until the PB2 OR enter is pressed.